Cross-Site Request Forgery Prevention Cheat Sheet

Introduction

A Cross-Site Request Forgery (CSRF) attack occurs when a malicious web site, email, blog, instant message, or program tricks an authenticated user's web browser into performing an unwanted action on a trusted site. If a target user is authenticated to the site, unprotected target sites cannot distinguish between legitimate authorized requests and forged authenticated requests.

Since browser requests automatically include all cookies including session cookies, this attack works unless proper authorization is used, which means that the target site's challenge-response mechanism does not verify the identity and authority of the requester. In effect, CSRF attacks make a target system perform attacker-specified functions via the victim's browser without the victim's knowledge (normally until after the unauthorized actions have been committed).

However, successful CSRF attacks can only exploit the capabilities exposed by the vulnerable application and the user's privileges. Depending on the user's credentials, the attacker can transfer funds, change a password, make an unauthorized purchase, elevate privileges for a target account, or take any action that the user is permitted to do.

In short, the following principles should be followed to defend against CSRF:

IMPORTANT: Remember that Cross-Site Scripting (XSS) can defeat all CSRF mitigation techniques! While Cross-Site Scripting (XSS) vulnerabilities can bypass CSRF protections, CSRF tokens are still essential for web applications that rely on cookies for authentication. Consider the client and authentication method to determine the best approach for CSRF protection in your application.

- **See the OWASP XSS Prevention Cheat Sheet for detailed guidance on how to prevent XSS flaws.**
- **First, check if your framework has built-in CSRF protection and use it**
- **If the framework does not have built-in CSRF protection, add CSRF tokens to all state-changing requests (requests that cause actions on the site) and validate them on the backend.**
- **If your software targets only modern browsers, you may rely on Fetch Metadata headers together with the fallback options described below to block cross-site state-changing requests.**

- **Stateful software should use the synchronizer token pattern**
- **Stateless software should use double submit cookies**
- **If an API-driven site can't use `<form>` tags, consider using custom request headers**
- **Implement at least one mitigation from Defense in Depth Mitigations section**
- **SameSite Cookie Attribute can be used for session cookies** but be careful to NOT set a cookie specifically for a domain. This action introduces a security vulnerability because all subdomains of that domain will share the cookie, and this is particularly an issue if a subdomain has a CNAME to domains not in your control.
- **Consider implementing user interaction based protection for highly sensitive operations**
- **Consider verifying the origin with standard headers**
- **Do not use GET requests for state changing operations.**
- **If for any reason you do it, protect those resources against CSRF**

Built-In Or Existing CSRF Implementations

Before building a custom token or Fetch-Metadata implementation, check whether your framework or platform already provides CSRF protection you can use. Built-in defenses are generally preferable because they're maintained by the framework authors and reduce the risk of subtle implementation mistakes. For example:

- .NET can use built-in protection to add tokens to CSRF vulnerable resources. If you choose to use this protection, .NET makes you responsible for proper configuration (such as key management and token management).
- Starting from 1.25, Go developers can rely on the built-in CrossOriginProtection type. It implements a Fetch-Metadata-based CSRF defense (including validation of Sec-Fetch-Site and related headers) directly in the standard library.

Token-Based Mitigation

The synchronizer token pattern is one of the most popular and recommended methods to mitigate CSRF.

Synchronizer Token Pattern

CSRF tokens should be generated on the server-side and they should be generated only once per user session or each request. Because the time range for an attacker to exploit the stolen tokens is

minimal for per-request tokens, they are more secure than per-session tokens. However, using per-request tokens may result in usability concerns.

For example, the "Back" button browser capability can be hindered by a per-request token as the previous page may contain a token that is no longer valid. In this case, interaction with a previous page will result in a CSRF false positive security event on the server-side. If per-session token implementations occur after the initial generation of a token, the value is stored in the session and is used for each subsequent request until the session expires.

When a client issues a request, the server-side component must verify the existence and validity of the token in that request and compare it to the token found in the user session. The request should be rejected if that token was not found within the request or the value provided does not match the value within the user session. Additional actions such as logging the event as a potential CSRF attack in progress should also be considered.

CSRF tokens should be:

- Unique per user session.
- Secret
- Unpredictable (large random value generated by a secure method).

CSRF tokens prevent CSRF because without a CSRF token, an attacker cannot create valid requests to the backend server.

Transmitting CSRF Tokens in Synchronized Patterns

The CSRF token can be transmitted to the client as part of a response payload, such as a HTML or JSON response, then it can be transmitted back to the server as a hidden field on a form submission or via an AJAX request as a custom header value or part of a JSON payload. A CSRF token should not be transmitted in a cookie for synchronized patterns. A CSRF token must not be leaked in the server logs or in the URL. GET requests can potentially leak CSRF tokens at several locations, such as the browser history, log files, network utilities that log the first line of a HTTP request, and Referer headers if the protected site links to an external site.

For example:

```
<form action="/transfer.do" method="post">
<input type="hidden" name="CSRFToken"
value="OWY4NmQwODE4ODRjN2Q2NTlhMmZlYWEwYzU1YWQwMTVhM2JmNGYxYjJiMGI4MjJjZDE1ZDZMGY
wMGEwOA==">
[...]
</form>
```

Since requests with custom headers are automatically subject to the same-origin policy, it is more secure to insert the CSRF token in a custom HTTP request header via JavaScript than adding a CSRF token in the hidden field form parameter.

ALTERNATIVE: Using A Double-Submit Cookie Pattern

If maintaining the state for CSRF token on the server is problematic, you can use an alternative technique known as the Double Submit Cookie pattern. This technique is easy to implement and is stateless. There are different ways to implement this technique, where the *naive* pattern is the most commonly used variation.

Signed Double-Submit Cookie (RECOMMENDED)

The most secure implementation of the Double Submit Cookie pattern is the *Signed Double-Submit Cookie*, which explicitly ties tokens to the user's authenticated session (e.g., session ID). Simply signing tokens without session binding provides minimal protection and remains vulnerable to cookie injection attacks. Always bind the CSRF token explicitly to session-specific data.

If the token contains sensitive information (like session IDs or claims), always use Hash-based Message Authentication (HMAC) with a server-side secret key. This prevents token forgery while ensuring integrity. HMAC is preferred over simple hashing in all cases as it protects against various cryptographic attacks. For scenarios requiring confidentiality of token contents, use authenticated encryption instead.

EMPLOYING HMAC CSRF TOKENS

To generate HMAC CSRF tokens (with a session-dependent user value), the system must have:

- **A session-dependent value that changes with each login session**. This value should only be valid for the entirety of the users authenticated session. Avoid using static values like the user's email or ID, as they are not secure (1 | 2 | 3). It's worth noting that updating the CSRF token too frequently, such as for each request, is a misconception that assumes it adds substantial security while actually harming the user experience (1). For example, you could choose one, or a combination, of the following session-dependent values:
  - The server-side session ID (e.g. PHP or ASP.NET). This value should never leave the server or be in plain text in the CSRF Token.
  - A random value (e.g. UUID) within a JWT that changes every time a JWT is created.
- **A secret cryptographic key** Not to be confused with the random value from the naive implementation. This value is used to generate the HMAC hash. Ideally, store this key as discussed in the Cryptographic Storage page.

- **A random value for anti-collision purposes**. Generate a random value (preferably cryptographically random) to ensure that consecutive calls within the same second do not produce the same hash (1).

Should Timestamps be Included in CSRF Tokens for Expiration?

It's a common misconception to include timestamps as a value to specify the CSRF token expiration time. A CSRF Token is not an access token. They are used to verify the authenticity of requests throughout a session, using session information. A new session should generate a new token (1).

PSEUDO-CODE FOR IMPLEMENTING HMAC CSRF TOKENS

Below is an example in pseudo-code that demonstrates the implementation steps described above:

```
// Gather the values
secret = getSecretSecurely("CSRF_SECRET") // HMAC secret key
sessionID = session.sessionID // Current authenticated user session
randomValue = cryptographic.randomValue(64) // Cryptographic random value

// Create the CSRF Token
message = sessionID.length + "!" + sessionID + "!" + randomValue.length + "!" +
randomValue.toHex() // HMAC message payload
hmac = hmac("SHA256", secret, message) // Generate the HMAC hash
// Add the `randomValue` to the HMAC hash to create the final CSRF token.
// Avoid using the `message` because it contains the sessionID in plain text,
// which the server already stores separately.
csrfToken = hmac.toHex() + "." + randomValue.toHex()

// Store the CSRF Token in a cookie
response.setCookie("csrf_token=" + csrfToken + "; Secure") // Set Cookie without
HttpOnly flag
```

Below is an example in pseudo-code that demonstrates validation of the CSRF token once it is sent back from the client:

```
// Get the CSRF token from the request
csrfToken = request.getParameter("csrf_token") // From header or form field (NOT
cookie)

// Split the token to get the randomValue
const tokenParts = csrfToken.split(".");
const hmacFromRequest = tokenParts[0];
const randomValue = tokenParts[1];

// Recreate the HMAC with the current session and the randomValue from the
request
```

```
secret = getSecretSecurely("CSRF_SECRET") // HMAC secret key
sessionID = session.sessionID // Current authenticated user session
message = sessionID.length + "!" + sessionID + "!" + randomValue.length + "!" +
randomValue

// Generate the expected HMAC
expectedHmac = hmac("SHA256", secret, message)

// Compare the HMAC from the request with the expected HMAC
if (!constantTimeEquals(hmacFromRequest, expectedHmac)) {
    // HMAC validation failed, reject the request
    response.sendError(403, "Invalid CSRF token")
    logError("Invalid CSRF token", hmacFromRequest, expectedHmac)
    return
}

// CSRF validation passed, continue processing the request
// ...
```

Note: The `constantTimeEquals` function should be used to compare the HMACs to prevent timing attacks. This function compares two strings in constant time, regardless of how many characters match.

Naive Double-Submit Cookie Pattern (DISCOURAGED)

> [!WARNING] The Naive Double-Submit Cookie pattern is bypassable by an attacker who can write cookies on the target domain (e.g., via a vulnerable sibling subdomain, DNS takeover, or plaintext-HTTP cookie injection on a non-`__Host-` cookie). For new code, use the Signed Double-Submit Cookie pattern above. The naive pattern is documented for reference only.

The *Naive Double-Submit Cookie* method is a scalable and easy-to-implement technique which uses a cryptographically strong random value as a cookie and as a request parameter (even before user authentication). Then the server verifies if the cookie value and request value match.

The site must require that every transaction request from the user includes this random value as a **custom request header or form parameter ONLY. Cookie validation is INSECURE**.

Why? Browsers auto-send cookies on cross-site requests. Attackers can trigger this automatically. Security requires *explicit* client submission (header/param) proving user intent.

If the value matches at server side, the server accepts it as a legitimate request and if they don't, it rejects the request.

Since an attacker is unable to access the cookie value during a cross-site request, they cannot include a matching value in the hidden form value or as a request parameter/header.

Though the Naive Double-Submit Cookie method is simple and scalable, it remains vulnerable to cookie injection attacks, especially when attackers control subdomains or network environments allowing them to plant or overwrite cookies. For instance, an attacker-controlled subdomain (e.g., via DNS takeover) could inject a matching cookie and thus forge a valid request token. This resource details these vulnerabilities. Therefore, always prefer the *Signed Double-Submit Cookie* pattern with session-bound HMAC tokens to mitigate these threats.

Fetch Metadata headers

Fetch Metadata request headers provide extra information about the context from which an HTTP request was made. Servers can use these headers — most importantly `Sec-Fetch-Site` — as a lightweight and reliable method to block obvious cross-site requests. See the Fetch Metadata specification for details.

Because some legacy browsers do not send `Sec-Fetch-*` headers, a fallback to standard origin verification headers **is a mandatory requirement** for any Fetch Metadata implementation. `Sec-Fetch-*` is supported in all major browsers since March 2023.

The Fetch Metadata request headers are:

- Sec-Fetch-Site — the primary signal for CSRF protection. It indicates the relationship between the request initiator's origin and its target's origin: `same-origin`, `same-site`, `cross-site`, or `none`.
- Sec-Fetch-Mode, Sec-Fetch-Dest, Sec-Fetch-User — additional headers that provide context about the request (such as the request mode, destination type, or whether it was triggered by a user navigation). More details are available in the MDN documentation.

If any of the headers above contain values not listed in the specification, in order to support forward-compatibility, servers should ignore those headers.

Ease of use

Unlike synchronizer tokens or double-submit patterns — which require additional client/server coordination and are difficult to implement correctly — Fetch Metadata checks are much more straightforward. They typically require only a small amount of server-side logic (inspect Sec-Fetch-Site, optionally refine with Sec-Fetch-Mode/Sec-Fetch-Dest) and no client changes. That simplicity reduces complexity, making the approach attractive for many applications.

Browser compatibility

Fetch Metadata request headers are supported in all modern browsers on both desktop and mobile (Chrome, Edge, Firefox, Safari 16.4+, and even in webviews on both iOS and Android), with over 98% global coverage. For compatibility detail, see the browser support table.

For the rare cases of outdated or embedded browsers that lack `Sec-Fetch-*` support, a fallback to standard origin verification should provide the required coverage. If this is acceptable for your project, consider prompting users to update their browsers, as they are running on outdated and potentially insecure versions.

How to treat Fetch Metadata headers on the server-side

`Sec-Fetch-Site` is the most useful Fetch Metadata header for blocking CSRF-like cross-origin requests and should be the primary signal in a Fetch-Metadata-based policy. Use other Fetch Metadata headers (`Sec-Fetch-Mode` , `Sec-Fetch-Dest` , `Sec-Fetch-User`) to further refine or tailor policies to your application's needs (for example, allowing top-level navigation requests or permitting specific Dest values for resource endpoints). **Policy (high level)**

1. If `Sec-Fetch-Site` is present

1.1. Treat cross-site as untrusted for state-changing actions. By default, reject non-safe methods (POST / PUT / PATCH / DELETE) when `Sec-Fetch-Site: cross-site` .

```
const SAFE_METHODS = new Set(['GET','HEAD','OPTIONS']);
const site = req.get('Sec-Fetch-Site'); // e.g. 'cross-site','same-site','same-
origin','none'

if (site === 'cross-site' && !SAFE_METHODS.has(req.method)) {
  return false; // forbid this request
}
```

1.2 If your application relies on safe HTTP methods (GET, HEAD, or OPTIONS) for state-changing actions, you should explicitly reflect that in your policy – e.g., by requiring a Fetch-Metadata header review for requests to those endpoints. This can be enforced with a policy rule like:

```
const SAFE_METHODS = new Set(['GET','HEAD','OPTIONS']);
const SENSITIVE_ENDPOINTS = new Set([
  '/user/profile',
  '/account/details',
]);

const site = req.get('Sec-Fetch-Site');
const path = req.path;

// Block if cross-site + unsafe method OR cross-site + sensitive endpoint
if (site === 'cross-site' && (!SAFE_METHODS.has(req.method) ||
```

```
SENSITIVE_ENDPOINTS.has(path))) {
  return false; // forbid this request
}
```

1.3. Allow `same-origin`. Treat `same-site` as allowed only if your threat model trusts sibling subdomains; otherwise handle `same-site` conservatively (for example, require additional validation).

```
const trustSameSite = false; // set true only if you trust sibling subdomains

if (site === 'same-origin') {
  return true;
} else if (site === 'same-site') {
  // handle same-site separately so the subcondition is clearly scoped to same-
site
  if (!trustSameSite && !SAFE_METHODS.has(req.method)) {
    return false; // treat same-site as untrusted for state-changing methods
  }
  return true;
}
```

1.4. Allow none for user-driven top-level navigations (bookmarks, typed URLs, explicit form submits) where appropriate.

2. If `Sec-Fetch-*` headers are absent: choose a fallback based on risk and compatibility requirements: 2.1. Fail-safe (recommended for sensitive endpoints): treat absence as unknown and block the request. 2.2. Fail-open (compatibility-first): fallback to (standard origin verification, CSRF tokens, and/or require additional validation).
3. Additional options

3.1 To ensure that your site can still be linked from other sites, you have to allow simple (HTTP GET) top-level navigation.

```
if (req.get('Sec-Fetch-Mode') === 'navigate' &&
    req.method === 'GET' &&
    req.get('Sec-Fetch-Dest') !== 'object' &&
    req.get('Sec-Fetch-Dest') !== 'embed') {
  return true; // Allow this request
}
```

3.2 Whitelist explicit cross-origin flows. If certain endpoints intentionally accept cross-origin requests (CORS JSON APIs, third-party integrations, webhooks), explicitly exempt those endpoints from the global Sec-Fetch deny policy and secure them with proper CORS configuration, authentication, and logging.

Requirements

- Your application must be served over trustworthy URLs. Fetch Metadata request headers are only sent to potentially trustworthy URLs. In practice, this includes `https`, `wss`, `file`, and `localhost` (including `127.0.0.0/8` and `::1/128`). See the W3C Secure Contexts spec for full details.
- HTTPS must be enforced across the entire application. This ensures consistent inclusion of Fetch Metadata headers. Enabling HTTP Strict Transport Security (HSTS) helps achieve this by automatically upgrading all HTTP requests to HTTPS.
- Safe HTTP methods should not be used for state-changing requests.

Concerns

- Prerender/prefetch and other speculative navigation may send `Sec-Fetch-*` values that don't match the final navigation, and browser-initiated flows (e.g., PaymentRequest) could generate requests without predictable fetch-metadata headers. These behaviors are still being refined, so header propagation isn't fully stable across all navigation types.
- Intermediaries (proxies, gateways, load balancers) may remove or modify `Origin` and `Sec-*` headers — whether due to privacy filters, network optimizations, or simple misconfiguration — which can break fetch-metadata-based protections. This kind of header stripping is problematic, but common.

Rollout & testing recommendations

- Include an appropriate `Vary` header, in order to ensure that caches handle the response appropriately. For example, `Vary: Sec-Fetch-Site, Origin`. See more Fetch Metadata specification.
  - Note that the `Vary` header does not impact CSRF defenses in any way. It is a response header, so it is applied after the server has already made its allow/deny decision based on CSRF protections. Its purpose is operational rather than defensive.
  - If the server responds differently based on HTTP headers (e.g., `Sec-Fetch-Site`, `Origin`), caches must vary on those headers. Without this, CDNs or proxies may reuse a response generated for a different context, causing broken behavior or contributing to cache-poisoning scenarios. Adding the appropriate `Vary` header ensures caches keep these responses separate.
- Start in "log only" mode. Record requests that would be blocked and review for false positives before enforcing. This is the safest way to discover legitimate flows that need whitelisting.

- Monitor UA coverage. Track which user agents include `Sec-Fetch-*` and which don't; ensure your fallback logic covers missing-header cases. Use metrics to decide when to enforce stricter policies.
- Document exceptions. Keep an explicit list of endpoints whitelisted for cross-origin access.

Disallowing simple requests

When a `<form>` tag is used to submit data, it sends a "simple" request that browsers do not designate as "to be preflighted". These "simple" requests introduce risk of CSRF because browsers permit them to be sent to any origin. If your application uses `<form>` tags to submit data anywhere in your client, you will still need to protect them with alternate approaches described in this document such as tokens.

> **Caveat:** Should a browser bug allow custom HTTP headers, or not enforce preflight on non-simple content types, it could compromise your security. Although unlikely, it is prudent to consider this in your threat model. Implementing CSRF tokens adds additional layer of defence and gives developers more control over security of the application.

Disallowing simple content types

For a request to be deemed simple, it must have one of the following content types - `application/x-www-form-urlencoded`, `multipart/form-data` or `text/plain`. Many modern web applications use JSON APIs so would naturally require CORS, however they may accept `text/plain` which would be vulnerable to CSRF. Therefore a simple mitigation is for the server or API to disallow these simple content types.

Employing Custom Request Headers for AJAX/API

Both the synchronizer token and the double-submit cookie are used to prevent forgery of form data, but they can be tricky to implement and degrade usability. Many modern web applications do not use `<form>` tags to submit data. A user-friendly defense that is particularly well suited for AJAX or API endpoints is the use of a **custom request header**. No token is needed for this approach.

In this pattern, the client appends a custom header to requests that require CSRF protection. The header can be any arbitrary key-value pair, as long as it does not conflict with existing headers.

```
X-CSRF-Token: RANDOM-TOKEN-VALUE
```

Many popular frameworks use standardized header names for CSRF protection:

- `X-CSRF-Token` - Ruby on Rails, Laravel, Django
- `X-XSRF-Token` - AngularJS
- `CSRF-Token` - Express.js (csurf middleware)
- `X-CSRFToken` - Django

While any arbitrary header name will work, using one of these standard names can improve compatibility with existing tools and developer expectations.

When handling the request, the API checks for the existence of this header. If the header does not exist, the backend rejects the request as potential forgery. This approach has several advantages:

- UI changes are not required
- no server state is introduced to track tokens

This defense relies on the CORS preflight mechanism which sends an `OPTIONS` request to verify CORS compliance with the destination server. All modern browsers designate requests with custom headers as "to be preflighted". When the API verifies that the custom header is there, you know that the request must have been preflighted if it came from a browser.

Custom Headers and CORS

Cookies are not set on cross-origin requests (CORS) by default. To enable cookies on an API, you will set `Access-Control-Allow-Credentials=true`. The browser will reject any response that includes `Access-Control-Allow-Origin=*` if credentials are allowed. To allow CORS requests, but protect against CSRF, you need to make sure the server only allows a few select origins that you definitively control via the `Access-Control-Allow-Origin` header. Any cross-origin request from an allowed domain will be able to set custom headers.

As an example, you might configure your backend to allow CORS with cookies from `http://www.yoursite.com` and `http://mobile.yoursite.com`, so that the only possible preflight responses are:

```
Access-Control-Allow-Origin=http://mobile.yoursite.com
Access-Control-Allow-Credentials=true
```

or

```
Access-Control-Allow-Origin=http://www.yoursite.com
Access-Control-Allow-Credentials=true
```

A less secure configuration would be to configure your backend server to allow CORS from all subdomains of your site using a regular expression. If an attacker is able to take over a subdomain (not uncommon with cloud services) your CORS configuration would allow them to bypass the same origin policy and forge a request with your custom header.

Dealing with Client-Side CSRF Attacks (IMPORTANT)

Client-side CSRF is a new variant of CSRF attacks where the attacker tricks the client-side JavaScript code to send a forged HTTP request to a vulnerable target site by manipulating the program's input parameters. Client-side CSRF originates when the JavaScript program uses attacker-controlled inputs, such as the URL, for the generation of asynchronous HTTP requests.

Note: These variants of CSRF are particularly important as they can bypass some of the common anti-CSRF countermeasures like token-based mitigations and SameSite cookies. For example, when synchronizer tokens or custom HTTP request headers are used, the JavaScript program will include them in the asynchronous requests. Also, web browsers will include cookies in same-site request contexts initiated by JavaScript programs, circumventing the SameSite cookie policies.

Client-Side vs. Classical CSRF: In the classical CSRF model, the server-side program is the most vulnerable component, because it cannot distinguish whether the incoming authenticated request was performed **intentionally**, also known as the confused deputy problem. In the client-side CSR model, the most vulnerable component is the client-side JavaScript program because an attacker can use it to generate arbitrary asynchronous requests by manipulating the request endpoint and/or its parameters. Client-side CSRF is due to an input validation problem and it reintroduces the confused deputy flaw, that is, the server-side won't, again, be able to distinguish if the request was performed intentionally or not.

For more information about client-side CSRF vulnerabilities, see Sections 2 and 5 of this paper, the CSRF chapter of the SameSite wiki, and this post by the Meta Bug Bounty Program.

Client-side CSRF Example

The following code snippet demonstrates a simple example of a client-side CSRF vulnerability.

```
<script type="text/javascript">
    const csrf_token = document.querySelector("meta[name='csrf-
token']").getAttribute("content");

    const ajaxLoad = () => {
        // process the URL hash fragment
        const hashFragment = window.location.hash.slice(1);
```

```
        // hash fragment should be of the format: /^(get|post);(.*)$/
        // e.g., https://site.com/index/#post;/profile
        if (hashFragment.length > 0 && hashFragment.includes(';')) {
            const params = hashFragment.match(/^(get|post);(.*)$/);

            if (params && params.length) {
                const requestMethod = params[1];
                const requestEndpoint = params[3];

                fetch(requestEndpoint, {
                    method: requestMethod,
                    headers: {
                        'X-CSRF-Token': csrf_token,
                        // [...]
                    },
                    // [...]
                })
                .then(response => { /* [...] */ })
                .catch(error => console.error('Request failed:', error));
            }
        }
    };

    // trigger the async request on page load - better practice is to use event
listeners
    window.addEventListener('DOMContentLoaded', ajaxLoad);
</script>
```

Vulnerability: In this snippet, the program invokes a function `ajaxLoad()` upon the page load, which is responsible for loading various webpage elements. The function reads the value of the URL hash fragment (line 4), and extracts two pieces of information from it (i.e., request method and endpoint) to generate an asynchronous HTTP request (lines 11-13). The vulnerability occurs in lines 15-22, when the JavaScript program uses URL fragments to obtain the server-side endpoint for the asynchronous HTTP request (line 15) and the request method. However, both inputs can be controlled by web attackers, who can pick the value of their choosing, and craft a malicious URL containing the attack payload.

Attack: Usually, attackers share a malicious URL with the victim (through elements such as spear-phishing emails) and because the malicious URL appears to be from an honest, reputable (but vulnerable) website, the user often clicks on it. Alternatively, the attackers can create an attack page to abuse browser APIs (e.g., the `window.open()` API) and trick the vulnerable JavaScript of the target page to send the HTTP request, which closely resembles the attack model of the classical CSRF attacks.

For more examples of client-side CSRF, see this post by the Meta Bug Bounty Program and this USENIX Security paper.

Client-side CSRF Mitigation Techniques

Independent Requests: Client-side CSRF can be prevented when asynchronous requests cannot be generated via attacker controllable inputs, such as the URL, window name, document referrer, and postMessages, to name only a few examples.

Input Validation: Achieving complete isolation between inputs and request parameters may not always be possible depending on the context and functionality. In these cases, input validation checks has to be implemented. These checks should strictly assess the format and choice of the values of the request parameters and decide whether they can only be used in non-state-changing operations (e.g., only allow GET requests and endpoints starting with a predefined prefix).

Predefined Request Data: Another mitigation technique is to store a list of predefined, safe request data in the JavaScript code (e.g., combinations of endpoints, request methods and other parameters that are safe to be replayed). The program can then use a switch parameter in the URL fragment to decide which entry of the list should each JavaScript function use.

Defense In Depth Techniques

SameSite (Cookie Attribute)

SameSite is a cookie attribute (similar to HTTPOnly, Secure etc.) which aims to mitigate CSRF attacks. It is defined in RFC6265bis. This attribute helps the browser decide whether to send cookies along with cross-site requests. Possible values for this attribute are `Lax`, `Strict`, or `None`.

The Strict value will prevent the cookie from being sent by the browser to the target site in all cross-site browsing context, even when following a regular link. For example, if a GitHub-like website uses the Strict value, a logged-in GitHub user who tries to follow a link to a private GitHub project posted on a corporate discussion forum or email, the user will not be able to access the project because GitHub will not receive a session cookie. Since a bank website would not allow any transactional pages to be linked from external sites, so the Strict flag would be most appropriate for banks.

If a website wants to maintain a user's logged-in session after the user arrives from an external link, SameSite's default Lax value provides a reasonable balance between security and usability. If the GitHub scenario above uses a Lax value instead, the session cookie would be allowed when following a regular link from an external website while blocking it in CSRF-prone request methods such as POST. Only cross-site-requests that are allowed in Lax mode have top-level navigations and use safe HTTP methods.

For more details on the `SameSite` values, check the following section from the rfc.

Example of cookies using this attribute:

```
Set-Cookie: JSESSIONID=xxxxx; SameSite=Strict
Set-Cookie: JSESSIONID=xxxxx; SameSite=Lax
```

All modern desktop and mobile browsers support the `SameSite` attribute. The main exceptions are legacy browsers including Opera Mini (all versions), UC Browser for Android, and older mobile browsers (iOS Safari < 13.2, Android Browser < 97). To track the browsers implementing it and know how the attribute is used, refer to the following service. Chrome implemented `SameSite=Lax` as the default behavior in 2020, and Firefox and Edge have followed suit. Additionally, the `Secure` flag is required for cookies that are marked as `SameSite=None`.

Limitations of SameSite

`SameSite` is useful as a defense-in-depth control but it does not replace a proper CSRF defense in most deployments. Treat the following as known gaps when reasoning about how much protection it actually provides:

- **`Lax` only blocks unsafe methods.** The default `Lax` behavior still allows the cookie on top-level navigations that use safe methods (`GET`, `HEAD`, `OPTIONS`, `TRACE`). If any state-changing operation in the application is reachable via a `GET` request, `SameSite=Lax` will not stop it. This is the single most common way `SameSite`-based defenses fail in practice. Review every `GET` endpoint and ensure that none of them mutate server-side state.
- **`SameSite` is scoped to the registrable domain, not the origin.** A cookie set on `app.example.com` with any `SameSite` value is still considered "same-site" when the request originates from `anything.example.com`. If your application shares a registrable domain with content you do not fully control (multi-tenant SaaS on a shared parent domain, subdomain-hosted user content, legacy subdomains, an acquired product running on the same parent domain, third-party services on subdomains), a vulnerability or malicious actor on any of those sibling hosts can issue requests that the browser will treat as same-site. This also amplifies the impact of subdomain takeovers: an attacker who claims a dangling subdomain can issue requests that your `SameSite`-protected cookies will accompany.
- **Top-level navigation and window-opening tricks.** An attacker who can get a victim to perform a top-level navigation or open a new window targeting your site (including through prerendering hints, `window.open`, or clicking a crafted link) can generate a request the browser treats as same-site. `SameSite=Strict` blocks most of these at the cost of breaking legitimate cross-site links into the app.

- **Browser coverage is not universal.** While current mainstream browsers enforce `SameSite=Lax` by default, users on older browsers, embedded browsers, or non-mainstream clients may receive cookies that behave as if no `SameSite` value were set. Do not assume all traffic enjoys the protection.
- **Client-side CSRF is unaffected.** `SameSite` operates on cross-site requests. It does not protect against client-side CSRF (see the earlier section on *Dealing with Client-Side CSRF Attacks*) where malicious input causes same-origin JavaScript in your own application to issue a state-changing request.

WHEN SAMESITE MAY BE SUFFICIENT ON ITS OWN

In narrow deployments `SameSite` alone can provide a reasonable CSRF defense, provided every one of the following holds:

- The application does not share a registrable domain with any host, subdomain, or service you do not fully control.
- No `GET` (or other safe-method) endpoint in the application performs a state-changing action. All state changes require `POST`, `PUT`, `PATCH`, or `DELETE`.
- The session cookie is set with `SameSite=Strict`, or `SameSite=Lax` combined with the `__Host-` prefix and a strict audit of every `GET` handler.
- Origin or Referer verification (see below) is in place for defense in depth on state-changing endpoints.
- You are comfortable excluding users whose browsers do not enforce `SameSite`, or you accept the residual risk that those users face.

For any application that does not meet all of the above, `SameSite` should be treated as a defense-in-depth layer and combined with a CSRF token or a double-submit pattern rather than relied on alone.

Using Standard Headers to Verify Origin

There are two steps to this mitigation method, both of which examine an HTTP request header value:

1. Determine the origin that the request is coming from (source origin). Can be done via Origin or Referer headers.
2. Determining the origin that the request is going to (target origin).

At server-side, we verify if both of them match. If they do, we accept the request as legitimate (meaning it's the same origin request) and if they don't, we discard the request (meaning that the

request originated from cross-domain). Reliability on these headers comes from the fact that they cannot be altered programmatically as they fall under forbidden headers list, meaning that only the browser can set them.

Identifying Source Origin (via Origin/Referer Header)

CHECKING THE ORIGIN HEADER

If the Origin header is present, verify that its value matches the target origin. Unlike the referer, the Origin header will be present in HTTP requests that originate from an HTTPS URL.

CHECKING THE REFERER HEADER IF ORIGIN HEADER IS NOT PRESENT

If the Origin header is not present, verify that the hostname in the Referer header matches the target origin. This method of CSRF mitigation is also commonly used with unauthenticated requests, such as requests made prior to establishing a session state, which is required to keep track of a synchronization token.

In both cases, make sure the target origin check is strong. For example, if your site is `example.org` make sure `example.org.attacker.com` does not pass your origin check (i.e, match through the trailing / after the origin to make sure you are matching against the entire origin).

If neither of these headers are present, you can either accept or block the request. We recommend **blocking**. Alternatively, you might want to log all such instances, monitor their use cases/behavior, and then start blocking requests only after you get enough confidence.

Identifying the Target Origin

Generally, it's not always easy to determine the target origin. You are not always able to simply grab the target origin (i.e., its hostname and port `#`) from the URL in the request, because the application server is frequently sitting behind one or more proxies. This means that the original URL can be different from the URL the app server actually receives. However, if your application server is directly accessed by its users, then using the origin in the URL is fine and you're all set.

If you are behind a proxy, there are a number of options to consider.

- **Configure your application to simply know its target origin:** Since it is your application, you can find its target origin and set that value in some server configuration entry. This would be the most secure approach as its defined server side, so it is a trusted value. However, this might be problematic to maintain if your application is deployed in many places, e.g., dev, test, QA, production, and possibly multiple production instances. Setting the correct value for each of these situations might be difficult, but if you can do it via some central configuration and provide your instances the ability to grab the value from it, that's great! (**Note:** Make sure the

centralized configuration store is maintained securely because major part of your CSRF defense depends on it.)

- **Use the Host header value:** If you want your application to find its own target so it doesn't have to be configured for each deployed instance, we recommend using the Host family of headers. The Host header is meant to contain the target origin of the request. But, if your app server is sitting behind a proxy, the Host header value is most likely changed by the proxy to the target origin of the URL behind the proxy, which is different than the original URL. This modified Host header origin won't match the source origin in the original Origin or Referer headers.
- **Use the X-Forwarded-Host header value:** To avoid the possibility that the proxy will alter the host header, you can use another header called X-Forwarded-Host to contain the original Host header value the proxy received. Most proxies will pass along the original Host header value in the X-Forwarded-Host header. So the value in X-Forwarded-Host is likely to be the target origin value that you need to compare to the source origin in the Origin or Referer header.

Using this header value for mitigation will work properly when origin or referrer headers are present in the requests. Though these headers are included the **majority** of the time, there are few use cases where they are not included (most of them are for legitimate reasons to safeguard users privacy/to tune to browsers ecosystem).

Use cases where X-Forward-Host is not employed:

- In an instance following a 302 redirect cross-origin, Origin is not included in the redirected request because that may be considered sensitive information that should not be sent to the other origin.
- There are some privacy contexts where Origin is set to "null" For example, see the following here.
- Origin header is included for all cross origin requests but for same origin requests, in most browsers it is only included in POST/DELETE/PUT **Note:** Although it is not ideal, many developers use GET requests to do state changing operations.
- Referer header is no exception. There are multiple use cases where referrer header is omitted as well (1, 2, 3, 4 and 5). Load balancers, proxies and embedded network devices are also well known to strip the referrer header due to privacy reasons in logging them.

Usually, a minor percentage of traffic does fall under above categories (1-2%) and no enterprise would want to lose this traffic. One of the popular technique used across the Internet to make this technique more usable is to accept the request if the Origin/referrer matches your configured list of domains "OR" a null value (Examples here. The null value is to cover the edge cases mentioned above where these headers are not sent). Please note that, attackers can exploit this but people

prefer to use this technique as a defense in depth measure because of the minor effort involved in deploying it.

Using Cookies with Host Prefixes to Identify Origins

While the `SameSite` and `Secure` attributes mentioned earlier restrict the sending of already set cookies and `HttpOnly` restricts the reading of a set cookie, an attacker may still try to inject or overwrite otherwise secured cookies (cf. session fixation attacks). Using `Cookie Prefixes` for cookies with CSRF tokens extends security protections against this kind of attacks as well. If cookies have `__Host-` prefixes e.g. `Set-Cookie: __Host-token=RANDOM; path=/; Secure` then each cookie:

- Cannot be (over)written from another subdomain and
- cannot have a `Domain` attribute.
- Must have the path of `/`.
- Must be marked as Secure (i.e, cannot be sent over unencrypted HTTP).

In addition to the `__Host-` prefix, the weaker `__Secure-` prefix is also supported by browser vendors. It relaxes the restrictions on domain overwrites, i.e., they

- Can have `Domain` attributes and
- can be overwritten by subdomains.
- Can have a `Path` other than `/`.

This relaxed variant can be used as an alternative to the "domain locked" `__Host-` prefix, if authenticated users would need to visit different (sub-)domains. In all other cases, using the `__Host-` prefix in addition to the `SameSite` attribute is recommended.

Cookie prefixes are supported by all major browsers.

See the Mozilla Developer Network and IETF Draft for further information about cookie prefixes.

User Interaction-Based CSRF Defense

While all the techniques referenced here do not require any user interaction, sometimes it's easier or more appropriate to involve the user in the transaction to prevent unauthorized operations (forged via CSRF or otherwise). The following are some examples of techniques that can act as strong CSRF defense when implemented correctly.

- Re-Authentication mechanisms
- One-time Tokens

Do NOT use CAPTCHA because it is specifically designed to protect against bots. It is possible, and still valid in some implementations of CAPTCHA, to obtain proof of human interaction/presence from a different user session. Although this makes the CSRF exploit more complex, it does not protect against it.

While these are very strong CSRF defenses, it can create a significant impact on the user experience. As such, they would generally only be used for security critical operations (such as password changes, money transfers, etc.), alongside the other defences discussed in this cheat sheet.

Possible CSRF Vulnerabilities in Login Forms

Most developers tend to ignore CSRF vulnerabilities on login forms as they assume that CSRF would not be applicable on login forms because user is not authenticated at that stage, however this assumption is not always true. CSRF vulnerabilities can still occur on login forms where the user is not authenticated, but the impact and risk is different.

For example, if an attacker uses CSRF to assume an authenticated identity of a target victim on a shopping website using the attacker's account, and the victim then enters their credit card information, an attacker may be able to purchase items using the victim's stored card details. For more information about login CSRF and other risks, see section 3 of this paper.

Login CSRF can be mitigated by creating pre-sessions (sessions before a user is authenticated) and including tokens in login form. You can use any of the techniques mentioned above to generate tokens. Remember that pre-sessions cannot be transitioned to real sessions once the user is authenticated - the session should be destroyed and a new one should be made to avoid session fixation attacks. This technique is described in Robust Defenses for Cross-Site Request Forgery section 4.1. Login CSRF can also be mitigated by including a custom request headers in AJAX request as described above.

REFERENCE: Sample JEE Filter Demonstrating CSRF Protection

The following JEE web filter provides an example reference for some of the concepts described in this cheatsheet. It implements the following stateless mitigations (OWASP CSRFGuard, cover a stateful approach).

- Verifying same origin with standard headers
- Double submit cookie
- SameSite cookie attribute

Please note that this is only a reference sample and is not complete (for example: it doesn't have a block to direct the control flow when origin and referrer header check succeeds nor it has a port/host/protocol level validation for referrer header). Developers are recommended to build their complete mitigation on top of this reference sample. Developers should also implement authentication and authorization mechanisms before checking for CSRF is considered effective.

Full source is located here and provides a runnable POC.

JavaScript: Automatically Including CSRF Tokens as an AJAX Request Header

The following guidance for JavaScript by default considers **GET**, **HEAD** and **OPTIONS** methods as safe operations. Therefore **GET**, **HEAD**, and **OPTIONS** method AJAX calls need not be appended with a CSRF token header. However, if the verbs are used to perform state changing operations, they will also require a CSRF token header (although this is a bad practice, and should be avoided).

The **POST**, **PUT**, **PATCH**, and **DELETE** methods, being state changing verbs, should have a CSRF token attached to the request. The following guidance will demonstrate how to create overrides in JavaScript libraries to have CSRF tokens included automatically with every AJAX request for the state changing methods mentioned above.

Storing the CSRF Token Value in the DOM

A CSRF token can be included in the `<meta>` tag as shown below. All subsequent calls in the page can extract the CSRF token from this `<meta>` tag. It can also be stored in a JavaScript variable or anywhere on the DOM. However, it is not recommended to store the CSRF token in cookies or browser local storage.

The following code snippet can be used to include a CSRF token as a `<meta>` tag:

```
<meta name="csrf-token" content="{{ csrf_token() }}">
```

The exact syntax of populating the content attribute would depend on your web application's backend programming language.

Overriding Defaults to Set Custom Header

Several JavaScript libraries allow you to override default settings to have a header added automatically to all AJAX requests.

XMLHttpRequest (Native JavaScript)

XMLHttpRequest's open() method can be overridden to set the `X-CSRF-Token` header whenever the `open()` method is invoked next. The function `csrfSafeMethod()` defined below will filter out the safe HTTP methods and only add the header to unsafe HTTP methods.

This can be done as demonstrated in the following code snippet:

```
<script type="text/javascript">
    const csrf_token = document.querySelector("meta[name='csrf-
token']").getAttribute("content");

    const csrfSafeMethod = (method) => {
        // these HTTP methods do not require CSRF protection
        return /^(GET|HEAD|OPTIONS)$/.test(method);
    };

    const originalOpen = XMLHttpRequest.prototype.open;
    XMLHttpRequest.prototype.open = function(...args) {
        const result = originalOpen.apply(this, args);

        if (!csrfSafeMethod(args[0])) {
            this.setRequestHeader('X-CSRF-Token', csrf_token);
        }

        return result;
    };
</script>
```

CSRF Prevention in modern Frameworks

Modern Single Page Application (SPA) frameworks like Angular, React, and Vue typically rely on the cookie-to-header pattern to mitigate Cross-Site Request Forgery (CSRF) attacks. This approach leverages the fact that browsers automatically attach cookies to cross-origin requests, but only JavaScript running on the same origin can read values and set custom headers—making it possible to detect and block forged requests. The cookie-to-header pattern works as follows:

1. Server generates a CSRF token: When a user authenticates or loads the app, the server sets a CSRF token in a cookie (e.g., `XSRF-TOKEN`). This cookie is accessible via JavaScript (i.e., not `HttpOnly`) and typically has `SameSite=Lax` or `Strict`.
2. Client reads the token: The SPA (often using a library like Angular's HttpClient or axios in React/Vue) reads the CSRF token from the cookie.
3. Client attaches the token to a custom header: For each state-changing request (`POST`, `PUT`, `DELETE`, etc.), the client sets the token as a custom HTTP header (commonly `X-XSRF-TOKEN` or `X-CSRF-TOKEN`).

4. Server validates the token: The server checks whether the token from the header matches the one from the cookie. If they match, the request is accepted; if not, it is rejected as potentially forged.

Angular provides this pattern out of the box, automatically handling steps 2 and 3 via its HttpClient. In contrast, frameworks like React and Vue require developers to implement this logic manually or with helper libraries such as axios interceptors. This pattern ensures that even if a browser includes cookies with a forged request, the attacker cannot set the matching custom header from another origin.

Angular

Angular's HttpClient supports the Cookie-to-Header Pattern used to prevent XSRF attacks. When performing HTTP requests, an interceptor reads a token from a cookie, by default `XSRF-TOKEN`, and sets it as an HTTP header, `X-XSRF-TOKEN`. Further documentation can be found at Angular's documentation for HttpClient XSRF/CSRF security.

```
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(withXsrfConfiguration({})),
    provideRouter(routes, withComponentInputBinding()),
  ],
};
```

This code snippet has been tested with Angular version 19.2.11.

React

For React applications, you can use axios interceptors to implement the cookie-to-header pattern:

```
// csrf-protection.js
import axios from 'axios';

// Function to get the CSRF token from cookies
const getCsrfToken = () => {
  const tokenCookie = document.cookie
    .split('; ')
    .find(cookie => cookie.startsWith('XSRF-TOKEN='));

  return tokenCookie ? tokenCookie.split('=')[1] : '';
};

// Create an axios instance with interceptors
const api = axios.create();

// Add a request interceptor to include the CSRF token in headers
```

```
api.interceptors.request.use(config => {
  // Only add for state-changing methods
  if (!/^(GET|HEAD|OPTIONS)$/i.test(config.method)) {
    config.headers['X-CSRF-Token'] = getCsrfToken();
  }
  return config;
});

export default api;
```

Axios

Axios allows us to set default headers for the POST, PUT, DELETE and PATCH actions.

```
<script type="text/javascript">
    const csrf_token = document.querySelector("meta[name='csrf-
token']").getAttribute("content");

    // Set CSRF token for state-changing methods
    axios.defaults.headers.post['X-CSRF-Token'] = csrf_token;
    axios.defaults.headers.put['X-CSRF-Token'] = csrf_token;
    axios.defaults.headers.delete['X-CSRF-Token'] = csrf_token;
    axios.defaults.headers.patch['X-CSRF-Token'] = csrf_token;

    // For TRACE method
    axios.defaults.headers.trace = {
        'X-CSRF-Token': csrf_token
    };

    // Alternative: Using interceptors for all requests
    axios.interceptors.request.use(config => {
        // Only add for state-changing methods
        if (!/^(GET|HEAD|OPTIONS)$/i.test(config.method)) {
            config.headers['X-CSRF-Token'] = csrf_token;
        }
        return config;
    });
</script>
```

This code snippet has been tested with Axios version 1.9.0.

jQuery

JQuery exposes an API called `$.ajaxSetup()` which can be used to add the `X-CSRF-Token` header to the AJAX request. API documentation for `$.ajaxSetup()` can be found here. The function `csrfSafeMethod()` defined below will filter out the safe HTTP methods and only add the header to unsafe HTTP methods.

You can configure jQuery to automatically add the token to all request headers by adopting the following code snippet. This provides a simple and convenient CSRF protection for your AJAX based applications:

```
<script type="text/javascript">
    const csrf_token = $('meta[name="csrf-token"]').attr('content');

    const csrfSafeMethod = method => {
        // these HTTP methods do not require CSRF protection
        return /^(GET|HEAD|OPTIONS)$/i.test(method);
    };

    $.ajaxSetup({
        beforeSend: (xhr, settings) => {
            if (!csrfSafeMethod(settings.type) && !settings.crossDomain) {
                xhr.setRequestHeader("X-CSRF-Token", csrf_token);
            }
        }
    });
</script>
```

This code snippet has been tested with jQuery version 3.7.1.

TypeScript Utilities for CSRF Protection

TypeScript allows you to create strongly typed utilities for CSRF protection. Here's a reusable utility module for CSRF token management:

```
// csrf-protection.ts

/**
 * Configuration options for CSRF protection
 */
interface CSRFOptions {
  /** Cookie name where the CSRF token is stored */
  cookieName: string;
  /** HTTP header name to use when sending the token */
  headerName: string;
  /** HTTP methods that require CSRF protection */
  unsafeMethods: string[];
}

/**
 * Default configuration for CSRF protection
 */
const DEFAULT_CSRF_OPTIONS: CSRFOptions = {
  cookieName: 'XSRF-TOKEN',
  headerName: 'X-CSRF-Token',
  unsafeMethods: ['POST', 'PUT', 'PATCH', 'DELETE']
```

```
};

/**
 * CSRF Protection utility class
 */
export class CSRFProtection {
  private options: CSRFOptions;

  constructor(options: Partial<CSRFOptions> = {}) {
    this.options = { ...DEFAULT_CSRF_OPTIONS, ...options };
  }

  /**
   * Extract CSRF token from cookies
   * @returns The CSRF token or empty string if not found
   */
  public getToken(): string {
    const cookieValue = document.cookie
      .split('; ')
      .find(cookie => cookie.startsWith(`${this.options.cookieName}=`));

    return cookieValue ? cookieValue.split('=')[1] : '';
  }

  /**
   * Check if the given HTTP method requires CSRF protection
   */
  public requiresProtection(method: string): boolean {
    return this.options.unsafeMethods.includes(method.toUpperCase());
  }

  /**
   * Add CSRF token to the provided headers object if needed
   */
  public addTokenToHeaders(method: string, headers: Record<string, string>):
Record<string, string> {
    if (this.requiresProtection(method)) {
      const token = this.getToken();
      if (token) {
        headers[this.options.headerName] = token;
      }
    }
    return headers;
  }
}

// Usage example:
// const csrfProtection = new CSRFProtection();
// const headers = csrfProtection.addTokenToHeaders('POST', {});
```

Angular with TypeScript

Angular is built with TypeScript, making it a natural fit for strongly-typed CSRF protection. The example below shows how to configure Angular's CSRF protection with TypeScript:

```
// app.config.ts
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withXsrfConfiguration } from '@angular/common/http';

import { routes } from './app.routes';

// Configure CSRF protection with custom options
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withXsrfConfiguration({
        cookieName: 'XSRF-TOKEN', // Name of cookie containing token
        headerName: 'X-XSRF-TOKEN' // Header name for token submission
      })
    ),
    provideRouter(routes)
  ]
};
```

For a custom HTTP interceptor that handles CSRF tokens:

```
// csrf.interceptor.ts
import { Injectable } from '@angular/core';
import {
  HttpRequest,
  HttpHandler,
  HttpEvent,
  HttpInterceptor
} from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class CsrfInterceptor implements HttpInterceptor {
  private readonly TOKEN_HEADER_NAME = 'X-CSRF-Token';
  private readonly SAFE_METHODS = ['GET', 'HEAD', 'OPTIONS'];

  constructor() {}

  intercept(request: HttpRequest<unknown>, next: HttpHandler):
Observable<HttpEvent<unknown>> {
    // Skip CSRF protection for safe methods
    if (this.SAFE_METHODS.includes(request.method)) {
      return next.handle(request);
    }

    // Get token from cookie
    const token = this.getTokenFromCookie();
```

```
    if (token) {
      // Clone the request and add the CSRF token header
      const modifiedRequest = request.clone({
        headers: request.headers.set(this.TOKEN_HEADER_NAME, token)
      });
      return next.handle(modifiedRequest);
    }

    return next.handle(request);
  }

  private getTokenFromCookie(): string {
    const tokenCookie = document.cookie
      .split('; ')
      .find(cookie => cookie.startsWith('XSRF-TOKEN='));

    return tokenCookie ? tokenCookie.split('=')[1] : '';
  }
}
```

React with TypeScript

Here's a TypeScript implementation for React applications using axios:

```
// csrf-axios.ts
import axios, { AxiosInstance, AxiosRequestConfig } from 'axios';

/**
 * Create an axios instance with CSRF protection
 */
export function createCSRFProtectedAxios(
  options: {
    baseURL?: string;
    csrfHeaderName?: string;
    csrfCookieName?: string;
  } = {}
): AxiosInstance {
  const {
    baseURL = '',
    csrfHeaderName = 'X-CSRF-Token',
    csrfCookieName = 'XSRF-TOKEN'
  } = options;

  // Create axios instance
  const instance = axios.create({ baseURL });

  // Add CSRF token interceptor
  instance.interceptors.request.use((config: AxiosRequestConfig) => {
    // Only add for non-GET requests
    if (config.method && !['get', 'head',
'options'].includes(config.method.toLowerCase())) {
```

```
      const token = getCsrfToken(csrfCookieName);

      if (token && config.headers) {
        config.headers[csrfHeaderName] = token;
      }
    }
    return config;
  });

  return instance;
}

/**
 * Extract CSRF token from cookies
 */
function getCsrfToken(cookieName: string): string {
  const tokenCookie = document.cookie
    .split('; ')
    .find(cookie => cookie.startsWith(`${cookieName}=`));

  return tokenCookie ? tokenCookie.split('=')[1] : '';
}

// USAGE EXAMPLE

// Define api.ts
// import { createCSRFProtectedAxios } from './csrf-axios';
// export const api = createCSRFProtectedAxios({
//   baseURL: '/api',
//   csrfHeaderName: 'X-CSRF-Token'
// });

// In a React component:
// import { api } from './api';
//
// function UserProfile() {
//   const updateUser = async (userData: UserData) => {
//     try {
//       // CSRF token is automatically added
//       const response = await api.post('/users/profile', userData);
//       return response.data;
//     } catch (error) {
//       console.error('Failed to update profile', error);
//     }
//   };
//
//   // Rest of component...
// }
```

For React applications using fetch API with TypeScript:

```
// csrf-fetch.ts

/**
 * Interface for CSRF protection options
 */
interface CSRFFetchOptions {
  csrfHeaderName: string;
  csrfCookieName: string;
  baseUrl: string;
}

/**
 * A wrapper around fetch API with CSRF protection
 */
export class CSRFProtectedFetch {
  private options: CSRFFetchOptions;

  constructor(options: Partial<CSRFFetchOptions> = {}) {
    this.options = {
      csrfHeaderName: 'X-CSRF-Token',
      csrfCookieName: 'XSRF-TOKEN',
      baseUrl: '',
      ...options
    };
  }

  /**
   * Performs a fetch request with CSRF protection
   */
  public async fetch<T>(
    url: string,
    options: RequestInit = {}
  ): Promise<T> {
    const { method = 'GET' } = options;
    const fullUrl = `${this.options.baseUrl}${url}`;

    // Create headers with CSRF token for unsafe methods
    const headers = new Headers(options.headers);

    if (!['GET', 'HEAD', 'OPTIONS'].includes(method.toUpperCase())) {
      const token = this.getCsrfToken();
      if (token) {
        headers.append(this.options.csrfHeaderName, token);
      }
    }

    // Perform request
    const response = await fetch(fullUrl, {
      ...options,
      headers
    });

    if (!response.ok) {
```

```
      throw new Error(`Request failed with status ${response.status}`);
    }

    return response.json();
  }

  /**
   * Shorthand for POST requests
   */
  public async post<T>(url: string, data: any, options: RequestInit = {}):
Promise<T> {
    return this.fetch<T>(url, {
      ...options,
      method: 'POST',
      body: JSON.stringify(data),
      headers: {
        ...options.headers,
        'Content-Type': 'application/json'
      }
    });
  }

  /**
   * Extract CSRF token from cookies
   */
  private getCsrfToken(): string {
    const tokenCookie = document.cookie
      .split('; ')
      .find(cookie => cookie.startsWith(`${this.options.csrfCookieName}=`));

    return tokenCookie ? tokenCookie.split('=')[1] : '';
  }
}

// USAGE EXAMPLE

// Create an instance
// const api = new CSRFProtectedFetch({
//   baseUrl: '/api',
//   csrfHeaderName: 'X-CSRF-Token'
// });
//
// // In React component
// const updateUser = async (userData: UserData) => {
//   try {
//     // CSRF token is automatically added
//     return await api.post('/users/profile', userData);
//   } catch (error) {
//     console.error('Failed to update profile', error);
//   }
// };
```

References in Related Cheat Sheets

CSRF

- OWASP Cross-Site Request Forgery (CSRF)
- PortSwigger Web Security Academy
- Mozilla Web Security Cheat Sheet
- Common CSRF Prevention Misconceptions
- Robust Defenses for Cross-Site Request Forgery
- For Java: OWASP CSRF Guard or Spring Security
- For PHP and Apache: CSRFProtector Project
- For Angular: Cross-Site Request Forgery (XSRF) Protection